



INFORMATION
TECHNOLOGY
UNIVERSITY

INTERNSHIP COMPLETION REPORT

Suricata IDS Monitoring, Rule Dataset & Attack Simulation

A live detection lab, a scraped rule dataset, a custom dashboard, and real malware traffic replayed end-to-end to test all of it.

NAME Sameeha Shahid

STUDENT ID BSCS23157

HOST ORGANIZATION Cyber Intelligence Lab (CIL), ITU

SUPERVISOR Fasih Ullah Khan Durrani

CO-SUPERVISOR Areeb Ahmed Bhutta

REPORTING PERIOD July 14 – August 28, 2026

Table of Contents

Introduction / Objectives	3
Tools & Technologies Used.....	3
Day-Wise Task & Results	3
Project Phases: Narrative Summary.....	5
Phase 1: Lab & Research Setup (Jul 14–17).....	5
Phase 2: Rule Scraping & Dashboard Foundation (Jul 20–31).....	5
Phase 3: Dashboard Optimization & PCAP Prep (Aug 3–12)	5
Phase 4: Attack Simulation & Debugging (Aug 13–20).....	5
Phase 5: Documentation & Finalization (Aug 24–28).....	5
Challenges Faced & Solutions.....	5
1. SQLite “database is locked” errors.....	6
2. Unbounded Suricata log growth (no rotation)	6
3. Dashboard not live-refreshing new alerts	6
Skills / Learning Outcomes	6
Network security & detection engineering.....	6
Data engineering.....	7
Full-stack application development.....	7
Debugging under real constraints	7
Final Deliverables.....	7
Conclusion	7
Appendix / Links	8
Project Repositories.....	8
Notion Notes.....	8
Google Drive.....	8
Reference Sources	8

Introduction / Objectives

This internship at the Cyber Intelligence Lab (CIL), ITU, was built around a single practical goal: stand up a working intrusion detection lab and use it to test detection under realistic conditions. The work spanned three linked objectives: deploying and validating a live Suricata IDS sensor across an isolated multi-VM network, building an independent rule metadata dataset covering a large number of signatures for context enrichment, and building a custom analyst dashboard from scratch to review, filter, and triage alerts. All three components were then tested together by replaying real, publicly documented malware traffic (sourced from malware-traffic-analysis.net) through the lab under multi-attacker, multi-victim conditions.

Tools & Technologies Used

- **Suricata 8.0.6**: network intrusion detection engine (core of the lab)
- **VMware Workstation**: isolated 3-VM lab (Kali attacker, Ubuntu-Benign target, Ubuntu-Suricata sensor)
- **Kali Linux, Ubuntu Server 26.04**: attacker and target/sensor operating systems
- **Python**: scraping, dataset generation, and dashboard backend logic
- **Streamlit**: the dashboard framework used to build the analyst UI from scratch
- **SQLite**: persistent storage for ingested alerts, replacing an in-memory-only prototype
- **Playwright + BeautifulSoup**: asynchronous web scraper for ET Open rule metadata (Proofpoint portal)
- **tcpdump, tshark, tcprewrite, tcpreplay, mergecap**: the packet capture / IP-remap / replay pipeline (the tcpreplay suite)
- **Apache2, vsftpd**: realistic target services running on Ubuntu-Benign for Suricata to observe
- **Git / GitHub**: version control and public documentation for both project repositories

Day-Wise Task & Results

The table below summarizes work on a day-by-day basis across the full reporting period.

Date(s)	Task	Result
Jul 14	Set up VMware lab: Kali (attacker), Ubuntu-Benign (victim), Ubuntu-Suricata (sensor) VMs	Working 3-VM lab with verified inter-VM connectivity
Jul 15	Explored ET Open and community Suricata rule sets	Understood rule syntax/structure needed for later rule-catalog work
Jul 16	Researched existing open-source Suricata dashboard solutions	Decided to build a custom Streamlit dashboard instead of adopting an existing tool
Jul 17	Researched Suricata rule categories, classtypes, and metadata fields	Documented reference notes used later for dashboard enrichment design

Date(s)	Task	Result
Jul 20–22	Began building a Playwright-based scraper for ET Open rule metadata (Proofpoint portal)	Initial batch of rule metadata successfully scraped into JSON
Jul 23–24	Initialized the Suricata dashboard project (Streamlit + SQLite)	Working repo scaffold ready for feature development
Jul 27	Built the eve.json log ingestion pipeline and SQLite schema	Alerts successfully parsed and stored in SQLite
Jul 28	Built the initial alert table UI and filters	Working, browsable live alert table in Streamlit
Jul 29	Continued dashboard UI work alongside scraper runs	More rules scraped; additional dashboard features added
Jul 31–Aug 3	Refined database schema and dashboard logic based on testing	Fixed schema issues; more stable ingestion and querying
Aug 5	Diagnosed slow dashboard rendering (repeated JSON re-parsing)	Implemented caching, significantly reduced render time
Aug 6	Documented performance findings and benchmarks	Written record of before/after optimization results
Aug 7–10	Sourced and tested a real malicious pcap (Nikto scan) from malware-traffic-analysis.net	Confirmed pcap suitable for lab replay
Aug 11	Ran a Nikto attack-replay simulation (tcprewrite, 5 disguised attacker IPs) against the lab victim	Verified Suricata generated correct alerts end-to-end, visible live on dashboard
Aug 12	Completed the full-scale ET rule scraping run	Finished suricata_et_rules.json rule catalog for dashboard enrichment
Aug 13–17	Built a combined 15-attacker-vs-5-victim simulation from 3 real attack pcaps (Formbook C2, web-probe scans, GuLoader/AgentTesla FTP exfil)	Verified alerts across all attacker/victim pairs; diagnosed and resolved log rotation, SQLite locking, and disk-space issues hit during testing
Aug 19–20	Rebuilt the simulation as a smaller, controlled 5-attacker-vs-5-victim set (2 Formbook, 2 webprobe, 1 ftpexfil)	Clean, verified replay with correctly labeled attacker IPs and confirmed live alerts
Aug 24–28	Wrote the full dashboard README and reorganized the GitHub repo structure	Public-facing documentation completed; repo restructured ahead of final submission

Project Phases: Narrative Summary

Phase 1: Lab & Research Setup (Jul 14–17)

Built the three-VM foundation for the entire project: a Kali attacker, an Ubuntu-Benign target running realistic services (Apache, vsftpd), and an Ubuntu-Suricata sensor, all bridged on an isolated VMnet1 host-only network (192.168.93.0/24) so lab traffic never touched the host's regular internet connection. Spent the remaining days researching Suricata's rule ecosystem and evaluating existing open-source dashboard tooling, which led to the deliberate decision to build a custom dashboard rather than adopt an off-the-shelf one.

Phase 2: Rule Scraping & Dashboard Foundation (Jul 20–31)

Started an automated Playwright-based scraper for ET Open rule metadata in parallel with building the dashboard's core: eve.json log ingestion, SQLite storage, and the first working alert table UI. This phase established the two main components, the rule scraper and the dashboard, that every later phase built on and refined.

Phase 3: Dashboard Optimization & PCAP Prep (Aug 3–12)

Diagnosed and fixed a performance bottleneck in the dashboard caused by repeated JSON re-parsing on every render, resolving it with caching and documenting the before/after benchmarks. Shifted focus to sourcing and validating real-world attack PCAPs (starting with a Nikto scan) for later simulation work, while finishing the full-scale ET rule scraping run covering roughly 70,000 signatures.

Phase 4: Attack Simulation & Debugging (Aug 13–20)

Built and replayed increasingly refined multi-attack simulations: first a larger 15-attacker-vs-5-victim scenario combining three real malware PCAPs (Formbook/XLoader C2, web reconnaissance probes, and GuLoader→AgentTesla FTP exfiltration), then a cleaner, more controlled 5-vs-5 version. This phase surfaced and resolved several genuine infrastructure issues, including Suricata log rotation, SQLite write-lock contention and a Streamlit live-refresh gap, each traced to its root cause and fixed rather than worked around.

Phase 5: Documentation & Finalization (Aug 24–28)

Closed out the internship by writing complete setup, architecture, and troubleshooting documentation for the dashboard, and reorganizing both GitHub repositories into a clear, dated structure ahead of final submission.

Challenges Faced & Solutions

Several of the most valuable parts of this internship came from debugging real infrastructure failures rather than following a clean, pre-planned path. The three issues below were each traced to a specific root cause and fixed directly, not worked around.

1. SQLite “database is locked” errors

- What happened: The dashboard intermittently threw a “database is locked” error during live ingestion.
- Root cause: The ingestion module's SQLite connection was using the default 5-second busy timeout, and a possible multi-tab Streamlit session could spawn duplicate ingestion threads writing to the same table concurrently.
- Fix: Gave one module sole ownership of writes to the alerts table, and applied a longer busy timeout with a threading lock around the ingestion routine to serialize concurrent writes safely.

2. Unbounded Suricata log growth (no rotation)

- What happened: The live dashboard appeared to show nothing during an actual replay test, despite Suricata clearly generating alerts.
- Root cause: eve.json and stats.log had grown unbounded (369MB+ and 231MB respectively) since initial setup, with no log rotation ever configured.
- Fix: Added Suricata's native daily rotate-interval plus a supporting logrotate policy (size-capped, 3 rotations retained, corrected file ownership permissions), keeping logs bounded going forward.

3. Dashboard not live-refreshing new alerts

- What happened: New alerts were visibly arriving in the eve.json log (confirmed via tail -f) but the Streamlit dashboard's alert table stayed static until manually reloaded.
- Root cause: Streamlit does not automatically re-run a page when new rows are written to the underlying database; it only re-runs on user interaction or an explicit refresh.
- Fix: Wrapped the alert table in a Streamlit fragment set to auto-run on a fixed interval, so the table refreshes on its own without a manual page reload.

Beyond these three, the PCAP replay pipeline itself surfaced a long list of smaller but instructive issues: capturing on the wrong network interface, oversized “super-packets” caused by NIC segmentation offload, missing destination MAC addresses causing silent Layer-2 frame drops, and a subtle IP-remapping bug that split a single real conversation into two unrelated fake identities, breaking Suricata's stream-reassembly logic. Each was diagnosed methodically using tshark and Suricata's own debug output before being fixed at the source.

Skills / Learning Outcomes

Network security & detection engineering

- Working knowledge of the Suricata rule engine: rule anatomy, SID/reference systems, fast-pattern matching, and how a packet moves from capture through decode, fast filter, full match, to alert.
- Hands-on packet capture and replay methodology using tcpdump, tshark, tcprewrite, and tcpreplay, including the subtleties of IP remapping and Layer 2 delivery in a virtualized network.

Data engineering

- Built a resilient, check pointed asynchronous scraper capable of resuming after interruption and recovering from a corrupted save file.
- Transformed unstructured, scraped web content into a clean, structured, query able JSON dataset covering tens of thousands of records.

Full-stack application development

- Took a dashboard from a memory-only prototype script to a persistent Streamlit + SQLite application with a real review workflow.
- Designed features (grouping, regex search, severity sorting, CSV export) around what an analyst actually needs when triaging a flood of alerts.

Debugging under real constraints

- Diagnosed concurrency issues (SQLite locking), infrastructure limits (disk exhaustion, unbounded logs), and UI framework quirks (Streamlit's refresh model) methodically rather than by guesswork.
- Learned to verify data integrity end-to-end (byte-for-byte sample checks, 0-failure verification runs) rather than assuming a pipeline worked simply because it completed.

Final Deliverables

The following are being submitted as the complete output of this internship:

- **GitHub repository:** suricata-dashboard, the dashboard codebase (app, ingestion, alert store, filters, charts, README)
- **GitHub repository:** suricata-ids-lab, lab setup, research notes, rule-dataset scripts, PCAP simulation work, and this presentation, organized into dated folders
- **Rule catalog dataset (suricata_et_rules.json):** ~70,000 scraped ET Open rule records with full metadata
- **Community rule sample dataset:** (suricata_community_sample.json)
- Captured and rewritten attack **PCAPs** used in the multi-attacker simulations (Formbook/XLoader, web-probe, GuLoader/AgentTesla FTP exfiltration, and the merged simulation files)
- Final project **presentation** (slide deck)
- **Google Drive** folder with the above large assets (datasets and PCAPs kept out of git)

Conclusion

This internship moved from a blank virtual lab to a working, end-to-end detection pipeline: a properly tuned Suricata sensor, a large rule metadata catalog built independently through scraping, and a custom-built dashboard capable of surfacing and triaging alerts generated by real malware traffic replayed under realistic multi-attacker conditions. Beyond the technical deliverables, some of the most valuable outcomes came from

the debugging process itself: tracing disk exhaustion, concurrency bugs, log rotation gaps, and UI refresh issues back to their root causes under real operating constraints.

Appendix / Links

Project Repositories

github.com/Sameeha135/suricata-dashboard

github.com/Sameeha135/suricata-ids-lab

Notion Notes

<https://believed-workshop-f82.notion.site/3a0c97a1a689806983dbd28741cfe5d9?v=3a0c97a1a68980cd946b000c9504a805&pvs=74>

Google Drive

<https://drive.google.com/drive/folders/1T34UG8KOCZISBnYT4ovgHsslwKf3OgWv?usp=sharing>

Reference Sources

malware-traffic-analysis.net: source of all replayed attack PCAPs

[Suricata official documentation \(OISF\)](#)

[Proofpoint Emerging Threats rule documentation \(ET Open rule details\)](#)